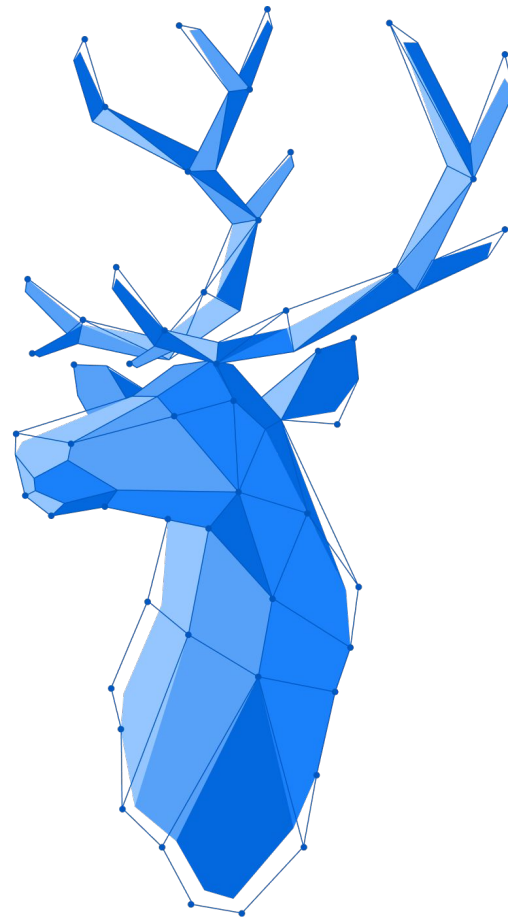




Lazy Loading Data Provider

By Gerald Koch



Konzept Lazy Loading

- Lazy loading bedeutet, dass man nur Dinge lädt, die man gerade benötigt - und alle anderen Dinge später wenn man sie benötigt. Z.B.: Ein Grid hat 15000 Zeilen, aber man sieht nur 50 - also laden wir zunächst 50 Zeilen und den Rest dann Schritt für Schritt beim Scrollen.
 - Reduziert Startzeit
 - Inhalte werden dynamisch während dem Scrollen geladen
- Vaadin Komponenten wie Grid, TreeGrid, Combobox besitzen schon automatisch Lazy Loading zwischen Client und Server, auch wenn 1000 Zeilen eingegeben werden
- Um die Server Seite zu beschleunigen, kann auch Lazy Loading eingeführt werden
- In Vaadin gibt es dafür Data Provider
- Es gibt 2 Methoden des Lazy Loadings
 - Mit Offset und Limit
 - Mit Seiten (Page)

Page vs. Limit and Offset

Page

- Wird durch Seitennummer und Seitengröße definiert
- Unterstützt von Spring Data
- Gut, wenn man nicht vorab wissen muss, wie groß die Gesamtmenge anfangs ist
- Ein All-in-One Objekt
- Eine methode im Data Provider
 - fetch

Offset and Limit

- Wird von Spring Data und Streams unterstützt
- Gut, wenn man vorab wissen möchte, wie groß die Gesamtmenge der Datensätze ist (z.B: wenn man die Anzahl der Datensätze anzeigen möchte)
- Näher an SQL
- 2 Methoden im Data Provider
 - fetch
 - count

Data Provider für Komponenten

Basis Data Provider

- `DataProvider<T, F>` → T = Datensatz Typ, F = Filter

`Grid<T>` unterstützt nativ keine Filter

- `DataProvider<T, Void>` → T = Datensatz Typ, kein Filter

`TreeGrid<T>`

- `HierarchicalDataProvider<T, ?>` → T = Datensatz Typ, erlaubt aber ignoriert Filter
- Zusätzliche Methoden für Parent-Child-Beziehung

`ComboBox<T>`

- `DataProvider<T, String>` → T = Datensatz Typ, String Filter

`VirtualList<T>`

- `DataProvider<T, ?>` → T = Datensatz Typ, erlaubt aber ignoriert Filter

Aber wie filtere ich ein Grid?

*** Schreib deinen eigenen Data Provider! ***

- Wenn man die Layer Isolation berücksichtigt, müssen Data Provider sowieso selbst geschrieben und zu Vaadin Data Providern konvertiert werden
- Den Pfad des Data Providers abfangen und Filter hinzufügen, welcher dann bis in die Persistenz mitgenommen wird, wo er evaluiert wird.
- Tip: Erstelle einen Adapter für die Vaadin Dataprovider für Grids, ComboBox, etc. in der UI Schicht
- Erstelle deine eigenen `ORDER BY` Klassen
- Schreibe Konverter für Sortierungs- und falls nötig für Paging Klassen
- Bitte daran Denken, dass der Komponenten Filter meist ein anderer Filter ist, als der Filter, den man verwenden möchte